Besoin Capturé

Réf : Groupe 7
09/02/2021
Étudiants Licence Mention Informatique 2

Contexte Client

M. Vincent Dugat, enseignant-chercheur à l'Université Toulouse III Paul-Sabatier ainsi que M. Gilles Lepinard, chef de projet à la société Thalès, souhaitent à terme développer une banque monétaire cryptographique « Bonoboland Central Bank » (BCB). Pour ce faire, M. Vincent Dugat souhaite utiliser une interface simple en langage C ou Java permettant d'effectuer des transactions simplifiées avec la cryptomonnaie « Bonobos » grâce au fonctionnement d'une blockchain. Dans ce cadre, il sollicite les étudiants de l'Université Toulouse III Paul-Sabatier pour concevoir et réaliser un programme prototype avec pour l'objectif plus global la validation du module « Projet S4 ».
M. Gilles Lepinard demande un suivi par étape de l'avancement du produit.
M. Vincent Dugat considère avoir identifié les exigences clefs relatives à la banque « BCB » et M. Gilles Lepinard permet un suivi approfondi du produit afin d'être au plus près des attentes. Ils comptent sur les étudiants pour concevoir et réaliser un prototype du programme qui réponde aux attentes.

Exigences Client

Catégorie	ID d'exigence	Titre Exigence	Criticité (Obligatoire/ Souhaitée/exemple)	Description Exigence	Méthode d'acceptation	Explication
Développement Projet	DP001		O	La blockchain centralisée doit comporter une structure de données	Programmation	**Détails page 9 du cahier des charges**
Développement Projet	DP002		O	Les blocs doivent comporter une structure de données	Programmation	**Détails page 10 du cahier des charges**
Développement Projet	DP003		O	Les listes de transactions simplifiées doivent comporter une structure de données	Programmation	**Détails page 10 du cahier des charges**
Développement Projet	DP004		O	Les blocs doivent être calculés par la méthode de chiffrement hash256	Programmation	**Fonction donnée en C, et package donné en Java**
Développement Projet	DP005		O	Les transactions doivent comporter une partie minage	Programmation	**Principe de la « nonce » introduit dans le contenu de chaque bloc**
Développement Projet	DP006		O	L'arbre de Merkle doit pouvoir passer les tests de validité de niveau 1 et 2	Programmation	**Détails page 6 du cahier des charges**
Développement Projet	DP007		O	L'arbre de Merkle doit comporter le bloc « Génésis »	Programmation	**Le premier bloc de la chaîne qui n'a pas de prédécesseur**
Développement Projet	DP008		O	Les hash des blocs doivent être cohérent en satisfaisant le critère	Programmation	**Critère <=> difficulté**
Développement Projet	DP009		O	Les racines de l'arbre de Merkle doivent être cohérent	Programmation	**Hash contenu dans le bloc avant de calculer son hash**
Développement Projet	DP010		O	L'arbre de Merkle doit pouvoir être calculé sur les listes de transactions simplifiées	Programmation	**Toute transaction simplifiée a un « Merkle tree root » (racines de l'arbre)**
Développement Projet	DP011		O	L'interface doit pouvoir permettre de lui donner des paramètres en ligne de commande et lancer le programme	Test proto	
Développement Projet	DP012		O	L'interface doit être composé d'une fonction main qui gère l'ensemble	Programmation	
Développement Projet	DP013		O	L'interface doit pouvoir récupérer en paramètre le niveau de difficulté pour générer la blockchain	Test proto	
Développement Projet	DP014		O	L'interface doit pouvoir récupérer en paramètre charge le nombre de blocs	Test proto	
Développement Projet	DP015		O	L'interface doit pouvoir récupérer en paramètre le nombre de transactions	Test proto	
Développement Projet	DP016		O	L'interface doit pouvoir récupérer en paramètre si utilisation ou non du cheater	Test proto	
Développement Projet	DP017		O	L'interface doit pouvoir récupérer en paramètre le numéro du bloc si utilisation du cheater	Test proto	
Développement Projet	DP018		S	L'interface doit pouvoir récupérer en paramètre le numéro de la transaction si utilisation du cheater	Test proto	
Développement Projet	DP019		O	Le cheater de bloc doit pouvoir récupérer en paramètre le numéro du bloc à supprimer	Test proto	
Développement Projet	DP020		O	Le cheater de transaction doit pouvoir récupérer en paramètre le numéro de la transaction à supprimer	Test proto	
Développement Projet	DP021		O	Le cheater de transaction doit pouvoir récupérer en paramètre le numéro du bloc associé à la transaction à supprimer	Test proto	
Développement Projet	DP022		O	Les blocs doivent contenir leur date de création	Programmation	**Fonction « getTimeStamp » en C**
Développement Projet	DP023		O	Le programme doit comporter un générateur aléatoire de transactions simplifiées	Programmation	**Génération aléatoire du nombre de Bnb**
Développement Projet	DP024		O	La version simplifiée du projet doit être implémentée en langage C et en langage Java	Programmation	
Développement Projet	DP025		O	La version améliorée du projet doit être implémentée au choix en langage C ou en langage Java	Programmation	
Développement Projet	DP026		O	Les structures de données en langage C doivent être dynamiques	Programmation	**Utilisation de pointeurs de transactions et pointeurs de blocs**
Développement Projet	DP027		O	Les transactions simplifiées doivent être sous forme de chaîne de caractères	Programmation	**Exemple : « Astérix envoie 10 Bnb à Obélix »**
Développement Projet	DP028		O	La nonce du bloc « Génésis » doit rester à zéro	Programmation	**Le hash de son prédécesseur du bloc « Génésis » est zéro**
Développement Projet	DP029		O	Chaque transaction doit contenir sa date	Programmation	**Fonction « getTimeStamp » en C**
Développement Projet	DP030		O	Le résultat d'un hash SHA256 doit être manipulé sous forme d'une chaîne de caractères hexadécimaux	Programmation	
Développement Projet	DP031		O	Le résultat d'un hash SHA256 doit être d'une longueur fixe	Programmation	**32 octets soit 64 caractères**
Développement Projet	DP032		O	La blockchain doit être constituée de blocs chaînés entre eux sous forme d'une liste	Programmation	**Utilisation de la structure de données « liste »**
Développement Projet	DP033		O	le choix de la difficulté doit être fixe	Programmation	**Une constante (entier)**
Développement Projet	DP034		O	Chaque bloc doit être signé	Programmation	**Ajout du hash de son contenu + celui de son prédécesseur**
Développement Projet	DP035		O	Le nombre de transactions doit être fixé	Programmation	**Une constante (entier)**
Développement Projet	DP036		O	La blockchain doit être mise en pratique avec une simulation de création monétaire par la BCB	Programmation	
Développement Projet	DP037		O	La partie simulation de création monétaire doit être dans un programme différent de celui du principe de la blockchain	Test proto	
Développement Projet	DP038		O	L'inflation de la BCB doit créer le « Génésis » avec une transaction de 50 BNB	Test proto	**Phase de création monétaire à partir de rien**
Développement Projet	DP039		O	Le programme doit comporter la phase « Helicopter Money »	Test proto	**Envoie de 50 BNB à tous les utilisateurs**
Développement Projet	DP040		O	Le programme doit s'arrêter par interruption de l'utilisateur ou le nombre de blocs a été atteint	Test proto	
Gestion de projet	GP001		O	La capture du besoin doit être rendu au plus tard le 9 février	Suivi	
Gestion de projet	GP002		O	La structuration du projet doit être rendu au plus tard le 16 mars	Suivi	
Gestion de projet	GP003		O	Le suivi du projet doit être rendu au plus tard le 5 mai	Suivi	
Gestion de projet	GP004		O	Le bilan du projet doit être rendu au plus tard le 11 mai	Bilan	